

Clase 4: Iteración + Introducción a listas

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas M Victoria Tolosa

WHILE LOOP



Iteración

Un loop es una estructura de control que permite repetir un cálculo/operatoria una cierta cantidad de veces.

While loop

En caso del while loop lo que determina la cantidad de iteraciones a dar es el hecho de cumplir una condición dada por una operación booleana.

Sintaxis

```
i=0
while i<=10:
    print(i)
    i=i+1
```

- **Indentación:** al igual que el if, una sangría delimita el rango de acción.
- **Corolario/guarda:** al igual que el if hay una operación cuyo resultado es un bool.
- **Contador:** no es condición necesaria para que el while exista. Pero usualmente los while loops tienen una variable que **cambia** dentro del mismo loop y que sirve para cortar la ejecución. En este caso es un contador.

Sintaxis

Un while loop seguirá corriendo siempre y cuando su corolario/guarda de como resultado True. Cuando dicho resultado pasa a ser False, la operación se corta y uno sale del loop

Ejemplos

```
while True:  
    print('Tres tristes tigres comen trigo en un trigal')
```

- Qué hace esta esta pieza de código? Es a propósito?

```
i=0  
while i<=10:  
    print(i)
```

```
i=0  
while i<=10:  
    print(i)  
    i=i+1
```

- Y aqui?

Iteración infinita

Los ejemplos anteriores dan cuenta de lo que sucede cuando no hay una manera de que la guarda de como resultado "False".

La mayoría de las veces (y en el marco de esta materia) esto es un bug.

Tip: a la hora de debuggear un script, los prints suelen ser muy útiles !

Ejercicio

Codear una function que reciba como input un número y devuelva si el mismo es un número primo o no.

Ejercicio 2

Así como en clases pasadas creamos una function recursiva que devuelva el factorial de un número. Crear ahora una function que devuelva nuevamente el factorial de un número pero iterando con un while loop.

LISTAS



Definiciones

Hasta ahora lidiamos simplemente con tipos de datos elementales.

Introducimos ahora el concepto de **estructuras de datos**. Una estructura de datos es un ítem/objeto capaz de almacenar varios datos en su interior. Los mismos pueden ser tipos de datos distintos (osea, dentro de una estructura de datos puede haber al mismo tiempo strings e integers por ejemplo).

En la materia veremos estos tipos de estructuras de datos:

- Listas
- Tuplas
- Sets
- Diccionarios

Listas

Estructura de datos que tiene las siguientes características:

- Sus elementos están ordenados de manera secuencial (esto las diferencia de los sets y los diccionarios)
- Sus elementos son mutables (esto las diferencia de las tuplas)
- Puede tener elementos repetidos (esto las diferencia de los sets)

Si bien las listas (al igual que el resto de las estructuras de datos) pueden tener dentro elementos de distinto tipo, esto se considera una **mala práctica**. Se espera que todos los elementos dentro de una lista tengan el mismo tipo.

Listas: sintaxis

- Las listas se representan con corchetes y sus elementos están separados por coma.
- Para crear una lista vacía se puede usar el comando `list()` o simplemente abrir y cerrar corchetes: `[]`

Ejemplos:

```
numeros=[1,2,3,4]
palabras=['hola','chau','adios']
listas= [[1,2,3],['hola','chau','adios']]
```

Qué notan en la 3ra variable?

OPERACIONES CON LISTAS

Operaciones

- **len(<lista>)**: Esta función devuelve un integer que da cuenta de la cantidad de elementos que esta lista posee.
- **<lista>.append(<element>)**: Esta función agrega un elemento a la última posición de la lista.
- **<lista>.insert(<posicion>, <element>)**: Agrega el elemento en la posición indicada.
- **<lista>.pop()**: Devuelve (y a la vez elimina de la lista) el elemento de la última posición de la misma.

Operaciones. Ejemplo

```
lista_ejemplo= [1,2,3,4,55]
last=lista_ejemplo.pop() ##Cuánto vale last? y lista_ejemplo?
last_2=lista_ejemplo.append(9) ##Cuánto vale last? y lista_ejemplo?
len(lista_ejemplo) #Cuánto vale esta operación
j=lista_ejemplo.insert(1,8) #Cuánto vale j? Y lista_ejemplo?
```

Muy importante: qué sucede en la asignación de last_2 y j?

Donde se inserta el 8 al usar insert?

Indexación

Como dijimos antes, una de las características de las listas es que sus elementos están ordenados en una secuencia.

Llamamos índice al lugar puntual de la lista donde ese elemento se encuentra.

Para referirnos a un elemento particular de una lista, la nomenclatura es `lista[indice]`

Esto nos puede servir para seleccionar un elemento, para redefinirlo o para borrarlo

Indexación. Ejemplo

Sigamos con el ejemplo anterior

```
lista_ejemplo= [1,2,3,4,55]

posi=lista_ejemplo[2] #Cuánto vale posi?
lista_ejemplo[3]=90 #Cuánto vale ahora lista_ejemplo?
del lista_ejemplo[0] #Y ahora?

lista_ejemplo[4]=9 #Esto corre?
```

Indexación. Aclaraciones

Como ya notamos en los ejemplos anteriores, en la notación de Python, **el primer número es el 0**. Esto aplica a todas las estructuras de datos que veremos en la materia que puedan ser indexadas.

Cuando pretendemos acceder de alguna manera a un elemento que no existe (pretendemos seleccionar el elemento 10 de una lista de 6 elementos) tendremos el error **index out of range**

Ejercicio FUNDAMENTAL

Dada la lista:

`lista_ej = [32,45,665,75,75,788,75,32]`

Generar un script que realice la sumatoria de los elementos de esta lista mediante iteración.

Ejercicio complementario

Utilizando nuevamente los conceptos de indexación e iteración, crear una function que nos diga si la siguiente lista está ordenada de mayor a menor:

```
lista=[7231,3232,321,33,2]
```

ALIASING: VALORS, OBJETOS Y REFERENCIAS

Valores y objetos

Hasta ahora venimos manejando los conceptos de valor y objeto como si fueran lo mismo. No lo son (precisamente)

Decimos que un objeto es básicamente una secuencia de bits que ocupan lugar en memoria y que pueden ser utilizados por python.

El valor es la expresión de ese objeto, la cual se guarda localmente.

Qué consecuencia tiene esta diferencia tan sutil?

QUE UN MISMO OBJETO PUEDE SER REPRESENTADO POR 2 VALORES. Y ahí pueden generarse confusiones.

Aliasing

Cuando hablamos de objetos y tipos de datos simples, objeto y valor son efectivamente sinónimos.

```
a=10  
b=10  
a=a-3  
  
b=a  
b=b-3
```

- a y b son 2 variables diferentes. Tienen el mismo valor pero son objetos diferentes
- A pesar de decir $a=b$, a y b siguen siendo 2 objetos diferenciados. Cada uno ocupa su lugar en memoria

Aliasing

```
lista_1=[1,2,3]
lista_2=[1,2,3]
lista_1.append(9)
#Cuanto valen lista_1 y lista_2?

#ATENCIÓN
lista_2=lista_1
lista_2.append(8)
#Cuánto valen ahora?
```

Pensemos estas preguntas por unos instantes

Aliasing

```
lista_1=[1,2,3]
lista_2=[1,2,3]
lista_1.append(9)
#Cuanto valen lista_1 y lista_2?

#ATENCION
lista_2=lista_1
lista_2.append(8)
#Cuánto valen ahora?
```

- Al principio está todo bien. lista_1 y lista_2 son 2 objetos creados independientes el uno del otro.
- Al variar uno, el otro no varía.
- Luego, usamos el operador asignación (=) y eso generó aliasing.
- lista_2=lista_1 quiere decir: tanto lista_1 como lista_2 apuntan al mismo objeto.
- Entonces, al modificar lista_2, lista_1 también se ve afectada.

Aliasing

- Este fenómeno se llama aliasing porque el objeto posee un nombre (lista_1 en nuestro ejemplo) y luego existe un **alias** que **referencia** el mismo objeto (lista_2).
- Por qué Python hace esto? Porque pretende ahorrar espacio. Un solo objeto con varios alias ocupa memoria una sola vez.
- No es muy difícil darse cuenta por que este comportamiento suele ser un motivo de bugs en el código. Hay que ser cuidadoso!

Ejercicio

Crear una function que tome como argumento una lista, appendee el número 2000 y retornee dicha lista. La lista input no debe ser modificada.

Tip

Si uno desea usar el operador asignación para crear una lista nueva que, además, sea un objeto diferente. Puede hacer un **deepcopy** con la función `list`.

```
lista_2= list(lista_1)
```

SLICING



Slicing

Es un operador que te permite obtener sublistas a partir de una lista.
El lado derecho del conjunto de un slicing no incluye el límite

```
lista=[1,2,3,9,8]

#[1,2]
lista_3=lista[0:2]

#[1,2,3]
lista_3=lista[:3]

#[3,9,8]
lista_3=lista[2:]

#[1,2,3,9]
lista_3=lista[:-1]

#[1,2,3,9,8]
lista_3=lista[:]

#[9,8]
lista_3=lista[-2:]
```

Ejercicio

Crear una function cuyo input sea una lista y su output otra lista distinta que sea el **reverso** de la lista input.